

Production Deployment Guide

Collateral Appraisal System — Windows Server / IIS release runbook

Contents

- | | |
|--|---|
| 1. Pre-deployment checklist | 6. IIS configuration (first-time commissioning) |
| Step 1 — Configure the application | 7. Post-deployment verification |
| Step 2 — Deploy the database (SQL scripts) | 8. Troubleshooting |
| Step 3 — Deploy the application per node | 9. Appendix — ports |

1. Pre-deployment checklist

Tick every box before the change window. The PowerShell one-liners below prove each item rather than assuming it.

1.1 Infrastructure reachable from both nodes

Run on: each app node ELEVATED POWERSHELL

```
# Replace the host names with the bank's real values.
Test-NetConnection SQLHOST -Port 1433 # SQL Server
Test-NetConnection REDISHOST -Port 6379 # Redis
Test-NetConnection localhost -Port 5672 # local RabbitMQ
Test-NetConnection SMTPHOST -Port 587 # mail relay
Test-NetConnection SFTPHOST -Port 22 # AS400 file drop
Test-NetConnection DCHOST -Port 389 # LDAP (636 if LDAPS)

# Each must print:
TcpTestSucceeded : True
```

Run on: each app node AS THE APP-POOL SERVICE ACCOUNT — NAS WRITE TEST

```
# Proves the SERVICE ACCOUNT (not your admin account) can write to the NAS.
Start-Process powershell -Credential (Get-Credential BANK\svc_cas) -ArgumentList @(
    '-NoProfile', '-Command',
    '"test" | Set-Content \\nas\CAS\uploads\_writetest.txt; Remove-Item \\nas\CAS\uploads\_writetest.txt'
)
```

- ☐ SQL Server reachable on `1433` from both nodes; a **fresh backup of the target database has been taken**.
- ☐ Redis reachable on `6379` from both nodes.
- ☐ RabbitMQ running **locally** on each node (`5672`) with the application user created.
- ☐ NAS / SMB share reachable and **writable by the app-pool identity** on both nodes.
- ☐ SMTP relay (`587`), SFTP (`22`) and AD/LDAP (`389` / `636`) reachable as required.
- ☐ The F5 can reach `https://<node>:7111/health/ready` and the SPA port on both nodes.

1.2 Security material (both nodes identical)

- ☐ OAuth2 **signing** certificate imported to `LocalMachine\My` ; app-pool identity has **Read** on the private key.
- ☐ OAuth2 **encryption** certificate imported to `LocalMachine\My` ; app-pool identity has **Read** on the private key.
- ☐ The **same two thumbprints** are configured on both nodes — verify with the command in §6.3.

1.3 Windows features

Run on: each app node ELEVATED POWERSHELL

```
Get-WindowsFeature Web-Server, Web-Static-Content | Format-Table Name, InstallState
# Expect InstallState = Installed for both

# ASP.NET Core Module v2 (from the .NET 9 Hosting Bundle)
Get-WebGlobalModule | Where-Object Name -like '*AspNetCore*'
# Expect: AspNetCoreModuleV2

# URL Rewrite (required by the SPA site)
Get-WebGlobalModule | Where-Object Name -eq 'RewriteModule'
# Expect one row. Empty output = URL Rewrite is NOT installed
```

Step 1 Configure the application (server-owned — authored once, edited later only when a value changes)

Generate `appsettings.Production.json` from `Bootstrapper/Api/appsettings.Production.json.template` and place it in `<APP_ROOT>\api`. The deploy script **never overwrites** this file (it is on the preserve list), so it survives every release. Substitute every `{TOKEN}` with:

Token	Setting	What to put
<code>#{DATABASE_CONNECTION_STRING}#</code>	<code>ConnectionStrings:Database</code>	SQL Server connection string for the <code>cas_app</code> login. Identical on both nodes.
<code>#{REDIS_CONNECTION_STRING}#</code>	<code>ConnectionStrings:Redis</code>	Shared Redis endpoint (cache + SignalR backplane).
<code>#{HANGFIRE_CONNECTION_STRING}#</code>	<code>ConnectionStrings:Hangfire</code>	SQL Server for the Hangfire job store — normally the same database.
<code>#{RABBITMQ_HOST}#</code>	<code>RabbitMQ:Host</code>	Local broker, e.g. <code>amqp://localhost:5672</code> .
<code>#{RABBITMQ_USERNAME}#</code> <code>#{RABBITMQ_PASSWORD}#</code>	<code>RabbitMQ:*</code>	Application user created per the RabbitMQ setup guide (not <code>guest</code>).
<code>#{OAUTH2_SIGNING_CERT_THUMBPRINT}#</code>	<code>OAuth2:SigningCertificate:Thumbprint</code>	Thumbprint of the signing certificate in <code>LocalMachine\My</code> (§6.3). Identical on both nodes.
<code>#</code> <code>{OAUTH2_ENCRYPTION_CERT_THUMBPRINT}#</code>	<code>OAuth2:EncryptionCertificate:Thumbprint</code>	Thumbprint of the encryption certificate. Identical on both nodes.
<code>#{APP_BASE_URL}#</code>	<code>AppBaseUrł</code>	Public API URL published by the F5 , e.g. <code>https://cas-api.lhbank.co.th</code> . Becomes the OpenIdDict issuer — must match on both nodes and must match what the SPA was built against.
<code>#{AUTH_INTERNAL_BASE_URL}#</code>	<code>Auth:InternalAuthBaseUrł</code>	Loopback URL for in-process token calls: <code>https://localhost:7111</code> — must match the site's scheme and port.
<code>#{ALLOWED_HOSTS}#</code>	<code>AllowedHosts</code>	Host-header allow-list, e.g. <code>cas-api.lhbank.co.th;localhost</code> , or <code>*</code> .
<code>#{FRONTEND_ORIGIN}#</code>	<code>Cors:AllowedOrigins</code>	The SPA origin , e.g. <code>https://cas.lhbank.co.th</code> . Must match exactly — scheme, host and port, no trailing slash — or the browser blocks every API call.
<code>#{FILESTORAGE_NAS_PATH}#</code>	<code>FileStorage:NasBasePath</code>	Shared UNC path, e.g. <code>\\nas\CAS</code> . Mode is <code>Nas</code> — required with two nodes.
<code>#{CHROMIUM_PATH}#</code>	<code>Reporting:ChromiumExecutablePath</code>	Full path to Edge or Chrome, e.g. <code>C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe</code> .
<code>#{ADMIN_USERNAME}#</code> <code>#{ADMIN_PASSWORD}#</code>	<code>SeedData:AdminUser</code>	Bootstrap administrator seeded on first boot. Change the password after the first sign-in.
<code>#{MAIL_HOST}#</code> <code>#{MAIL_USERNAME}#</code> <code>#{MAIL_PASSWORD}#</code> <code>#</code> <code>{MAIL_FROM_ADDRESS}#</code>	<code>Mail:*</code>	SMTP relay (port 587, STARTTLS) and sender address.
<code>#{AS400_SFTP_HOST}#</code> <code>#{AS400_SFTP_USERNAME}#</code> <code>#{AS400_SFTP_PASSWORD}#</code>	<code>FileTransfer:Inbound/Outbound:Sftp</code>	AS400 reappraisal file-drop credentials.

The **LDAP** block ships pre-filled for the bank domain (`Ldap:Enabled = true` , `Server = LHB.NET` , `Domain = LHB` , integrated authentication). Fill `BindDn` / `BindPassword` only if a service-account bind is required instead of integrated auth.

Step 2 Deploy the database (DBA — ONCE per release, BEFORE any application node is deployed)

What the `db\` folder contains

#	File	What it does	Approx. size
1	00_Prepere.sql	Creates <code>dbo.DatabaseMigrationHistory</code> , the journal table that records which scripts have run.	1 KB
2	01_EF_01_CommonDbContext.sql ... through ... 01_EF_11_ReportingDbContext.sql	Tables, keys, indexes for all 11 modules. Each file is an EF Core <i>idempotent</i> script: every migration is wrapped in <code>IF NOT EXISTS (SELECT ... FROM __EFMigrationsHistory)</code> , so already-applied migrations are skipped. The numbering is the required run order — module dependencies rely on it.	7 KB – 690 KB (~1.1 MB total)
3	02_Repeatable_ViewsAndProcs.sql	All 61 views and stored procedures, emitted in dependency order. Each is <code>CREATE OR ALTER</code> , so re-running always leaves the current definition in place.	~250 KB
4	03_OneTime_DataScripts.sql	39 one-time reference-data and data-fix scripts (Thai province/district/sub-district tables, parameters, document types, pricing templates, meeting configuration, ...). Each block is skipped if the journal already records it.	~1.9 MB
5	99_Verify.sql	Read-only. Run after the others and read the output.	2 KB

How re-running is made safe. Three independent mechanisms, all standard:

- **EF migrations** — recorded in `__EFMigrationsHistory` inside each module's schema (`request`, `workflow`, `appraisal`, ...). The generated script tests that table before every migration.
- **Views and stored procedures** — `CREATE OR ALTER`: no drop, no data loss, correct result whether the object exists or not.
- **One-time data scripts** — each block starts with a lookup in `dbo.DatabaseMigrationHistory`. If the script is already recorded, the block turns on `SET NOEXEC ON` so SQL Server parses but does not execute the statements, then turns it off again and moves on. The journal row is written at the end of the block.

Consequence: if a run fails halfway, fix the cause and **run the same file again**. Everything already applied is skipped.

Method A — SQL Server Management Studio

Best when the DBA wants to review each script before executing it.

SSMS ▶ Connect to Server ▶ File ▶ Open ▶ File... ▶ Execute (F5)

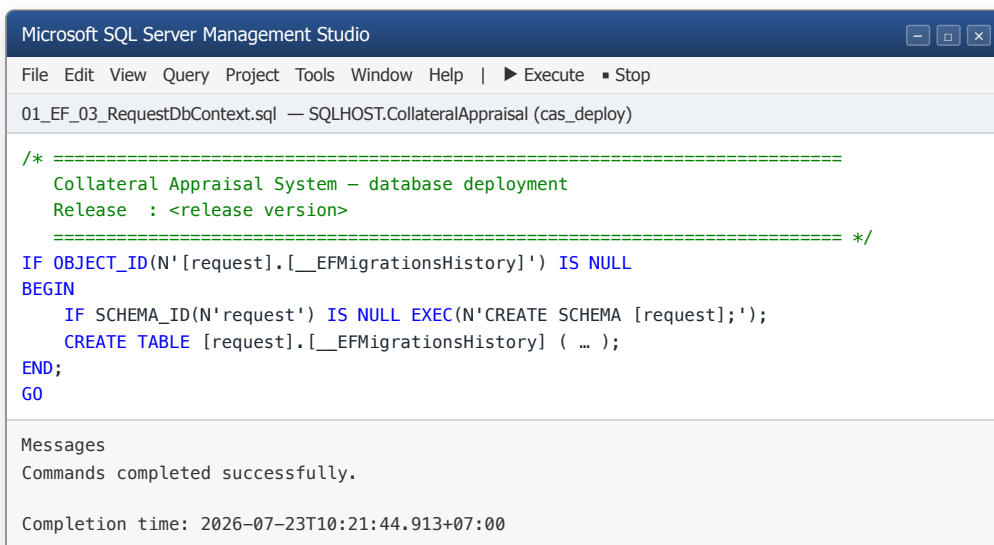


Figure 4.3 — SSMS with a deployment script open. Confirm the status bar shows the **correct server and database** before pressing F5. Results appear on the **Messages** tab, not the Results grid.

1. Connect to the SQL instance as `cas_deploy`.
2. **File ▶ Open ▶ File...** and open `00_Prepere.sql`.
3. Confirm the database dropdown in the toolbar shows **CollateralAppraisal** (not `master`).
4. Press **F5**. Read the **Messages** tab.

- Repeat for `01_EF_01_...` through `01_EF_11_...` in numeric order, then `02_Repeatable_ViewsAndProcs.sql`, then `03_OneTime_DataScripts.sql`.
- Finally open `99_Verify.sql` and review its output (see Verify the database deployment).

Method B — sqlcmd

Faster, scriptable, and produces a log file per script — the better choice for a change record.

Run on: SQL host or a jump box with sqlcmd COMMAND PROMPT / POWERSHELL

```
# -b : stop with a non-zero exit code on the first SQL error
# -V 16 : treat severity 16 and above as failure
# -E : Windows authentication (use -U/-P for a SQL login)
cd <STAGING_ROOT>\db

sqlcmd -S SQLHOST -d CollateralAppraisal -E -b -V 16 -i 00_Prepate.sql -o 00_Prepate.log
sqlcmd -S SQLHOST -d CollateralAppraisal -E -b -V 16 -i 01_EF_01_CommonDbContext.sql -o 01_EF_01.log
sqlcmd -S SQLHOST -d CollateralAppraisal -E -b -V 16 -i 01_EF_02_ParameterDbContext.sql -o 01_EF_02.log
... continue in numeric order through 01_EF_11_ReportingDbContext.sql ...
sqlcmd -S SQLHOST -d CollateralAppraisal -E -b -V 16 -i 02_Repeatable_ViewsAndProcs.sql -o 02.log
sqlcmd -S SQLHOST -d CollateralAppraisal -E -b -V 16 -i 03_OneTime_DataScripts.sql -o 03.log -t 1800
```

Or run the whole sequence with the supplied wrapper, which enforces the order, aborts on the first error and writes one log per file:

Run on: SQL host or a jump box POWERSHELL

```
.\deploy\Invoke-SqlDeploy.ps1 -ServerInstance SQLHOST -Database CollateralAppraisal `
-ScriptPath <STAGING_ROOT>\db -TrustedConnection

# Dry run first - lists what would execute without touching the database:
.\deploy\Invoke-SqlDeploy.ps1 -ServerInstance SQLHOST -Database CollateralAppraisal `
-ScriptPath <STAGING_ROOT>\db -TrustedConnection -WhatIf
```

Expected output

```
Database deploy
server : SQLHOST
database: CollateralAppraisal
scripts: <STAGING_ROOT>\db (13 files)
-> 00_Prepate.sql
    ok (1s)
-> 01_EF_01_CommonDbContext.sql
    ok (3s)
...
-> 02_Repeatable_ViewsAndProcs.sql
    ok (22s)
-> 03_OneTime_DataScripts.sql
    ok (96s)

Database deployed. Now run 99_Verify.sql and review its output.
```

Inside the logs, `03_OneTime_DataScripts.log` shows one line per script — this is how you see what a re-run skipped:

```
APPLY: 20260719140000_SeedData_VesselTypeParameter.sql
APPLY: 20260720120000_SeedData_FireInsuranceRates.sql
SKIP (already applied): 20250804170600_Initial.sql
SKIP (already applied): 20260311162600_SeedData_DopaProvinces.sql
```

Rules

Four rules that must not be broken.

- **Run once per release, from one place.** Never run the scripts concurrently from two sessions.
- **Run before deploying the application.** A node that starts against an un-migrated database will either try to migrate it itself (if it has DDL rights) or fail to start.
- **Keep the numeric order.** `01_EF_01` creates tables that `01_EF_03` references; `02` creates views over those tables; `03` populates them.
- **There is no automatic schema rollback.** Schema changes within a release are backward-compatible so the previous application build keeps running during the rollout, but reversing a migration means restoring the pre-release backup.

Verify the database deployment

Run on: SQL host READ-ONLY

```
sqlcmd -S SQLHOST -d CollateralAppraisal -E -i 99_Verify.sql -o 99_Verify.log
notepad 99_Verify.log
```

Check in <code>99_Verify.sql</code>	Expected result
1. EF migrations applied per history table	One row per module schema (<code>appraisal</code> , <code>auth</code> , <code>collateral</code> , <code>common</code> , <code>dbo</code> , <code>document</code> , <code>integration</code> , <code>notification</code> , <code>reporting</code> , <code>request</code> , <code>workflow</code>), each with a non-zero count.
1b. Stray history tables	Zero rows. A row means an <code>__EFMigrationsHistory</code> table landed outside the module schemas — usually because the deployment login's default schema is not <code>dbo</code> . Stop and fix it before releasing.
2. Journal totals	<code>RepeatableObjects</code> ≥ 61 , <code>OneTimeScripts</code> ≥ 39 , <code>LastApplied</code> = today.
3. Objects present	<code>VIEW</code> ≥ 55 and <code>SQL_STORED_PROCEDURE</code> ≥ 2 .
4. Unresolved references	Zero rows. Any row means a view or procedure references an object that does not exist — investigate before releasing.
5. Reference-data spot check	<code>DopaProvinces</code> ≈ 77 , <code>DopaDistricts</code> and <code>DopaSubDistricts</code> in the thousands.
6. Release stamp	The most recent rows all carry the release version of the scripts you just ran.

Step 3 Deploy the application per node (first installation — manual file copy)

The application is a folder of files: deploying it means **copying two folders into place**. There is no installer and no deployment program. On a first installation the target folders are empty, so this is a straight copy — nothing to back up, nothing to remove.

Do §6 first. The application pool, the two websites and the certificates must exist before the files are of any use. If §6 has not been done on this node yet, do it now and come back.

3.1 Backend — `<APP_ROOT>\api`

1. **Confirm the pool is stopped.** `IIS Manager` $\triangleright$ `Application Pools` $\triangleright$ `CAS` $\triangleright$ `Stop` Not strictly required with no application running yet, but it keeps the node from serving a half-copied folder.
2. **Copy the build in.** Copy the entire contents of `<STAGING_ROOT>\api\` into `<APP_ROOT>\api` .
3. **Put the configuration file in place.** Copy the `appsettings.Production.json` you prepared in Step 1 into `<APP_ROOT>\api` . The release bundle deliberately does not contain it.
4. **Confirm the folder contents.** `Api.dll` , `web.config` and `appsettings.Production.json` are all present.
5. **Create the `logs` folder** if it is not there, and confirm the app-pool service account has **Modify** on it — otherwise the application starts but writes no log, which makes every later problem harder to diagnose.

The upload limit is applied to this `web.config` . The file that just arrived with the build is the one §6.5 edits. Apply §6.5 **after** this copy, not before — otherwise the copy overwrites the change.

3.2 Frontend — <APP_ROOT>\web

- 1. Copy the entire contents of <STAGING_ROOT>\web\ into <APP_ROOT>\web , including web.config — unlike the API, the SPA's web.config comes from the release, because it carries the routing rules that make deep links work.
- 2. Confirm index.html , web.config and assets\ are all present.

3.3 Start the node and confirm it is healthy

- 1. Start the application pool. IIS Manager > Application Pools > CAS > Start Both websites share this pool, so one start covers the API and the SPA.
- 2. The application starts on the first request. That first request takes noticeably longer than later ones: on a brand-new database it also seeds menus, permissions, roles, the OAuth2 client applications and the workflow, SLA and committee configuration. Give it a minute before deciding something is wrong.

Then check the two addresses on the server itself:

Address	Expected
https://localhost:7111/health/ready	Loads and reports Healthy . This is what the F5 uses to decide whether the node takes traffic.
https://localhost:3000/	The application's sign-in page loads.

Run on: the node just deployed COMMAND LINE ALTERNATIVE

```
# -k skips certificate validation: the cert is issued for the public host name, not localhost.
curl.exe -k -s -o NUL -w "%{http_code}\n" https://localhost:7111/health/ready
# Expect 200. If it is not 200, see §7 for how to find out which dependency failed.
```

If the health check does not come back within a minute or two, do not move on to the second node. Read <APP_ROOT>\api\logs\log-*.txt and the Windows Event Log (§8.1) — the first start is where a wrong connection string, a missing certificate permission or an unreachable dependency shows up.

3.4 Then the second node

Repeat 3.1 to 3.3 on node 2, using the same release folder and the same appsettings.Production.json values — the certificate thumbprints, database connection string and AppBaseUrl must be identical on both nodes, or tokens issued by one node are rejected by the other.

Later releases work differently. Once a build is live, an upgrade is not a copy over the top: the old files must be removed first, keeping appsettings.Production.json , web.config and logs\ . Leaving old DLLs behind mixes two builds and produces errors such as TypeLoadException or MissingMethodException , often on one screen and hours later. Take a copy of the live folders before overwriting them, so the previous build can be put back.

6. IIS configuration (first-time server commissioning — one-time per node)

These steps are performed once per server. Each has both a UI path and a PowerShell equivalent; use whichever the bank's change process prefers. The figures are faithful recreations of the real dialogs.

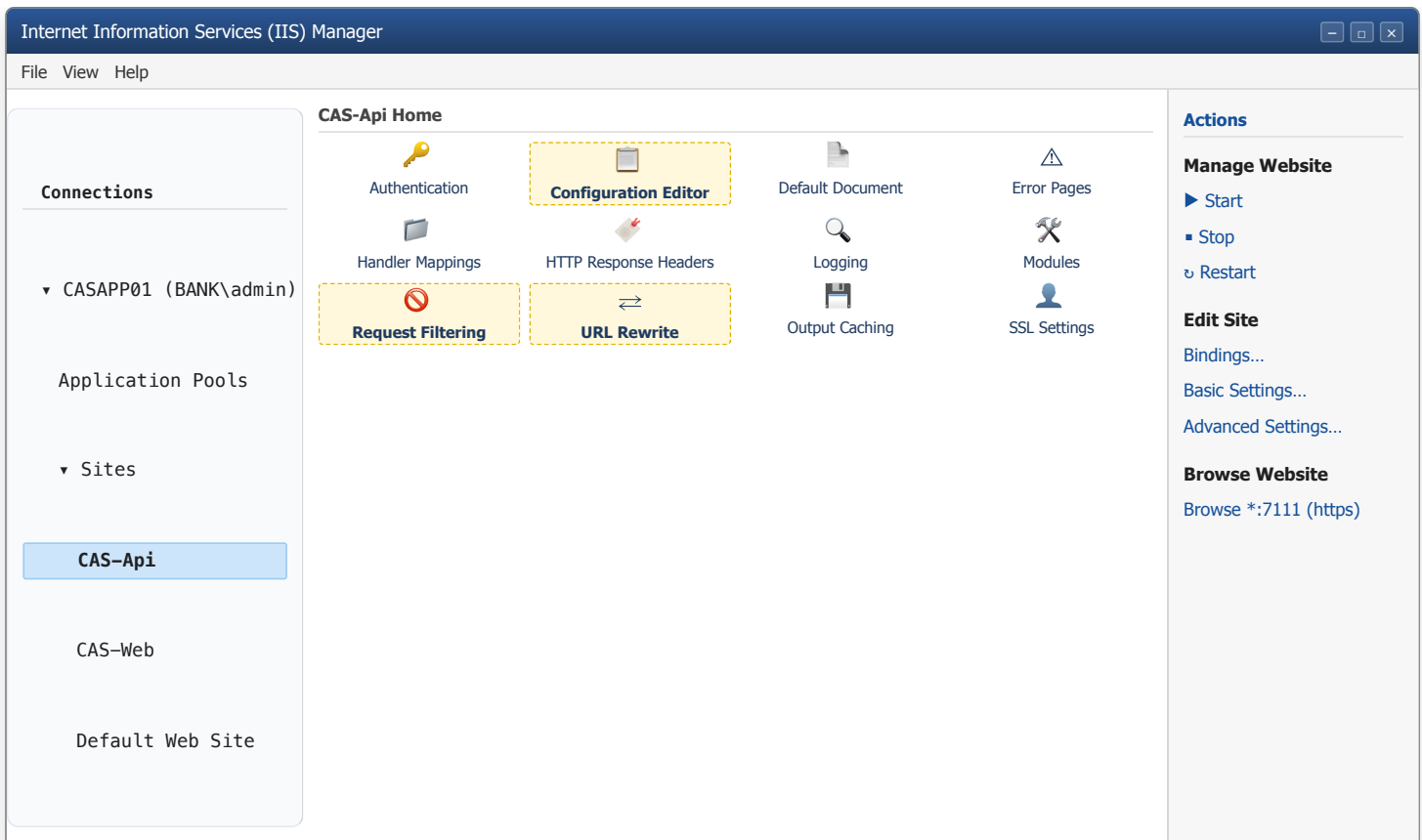


Figure 6.0 — IIS Manager after commissioning: two sites (`CAS-Api` , `CAS-Web`). The highlighted feature icons are the ones this section uses. Open with `inetmgr` or **Server Manager** > **Tools** > **Internet Information Services (IIS) Manager**.

6.1 Create the application pools

A single application pool named `CAS` hosts **both** websites — the API and the SPA. It uses **No Managed Code**: the API self-hosts .NET through ANCM, and the SPA is static files.

One pool means one worker process for both sites. That keeps configuration simple and gives both sites the same identity and the same NAS and certificate access. The trade-off to remember at deployment time: **stopping the pool takes the SPA down as well as the API** — requests to the SPA return HTTP 503 while it is stopped. Step 3 is written around that.

IIS Manager > Application Pools > Add Application Pool...

Figure 6.1 — Add Application Pool. .NET CLR version = `No Managed Code`; pipeline stays `Integrated`. One pool serves both websites.

Run on: each app node POWERSHELL EQUIVALENT

```

Import-Module WebAdministration

New-WebAppPool -Name 'CAS'
Set-ItemProperty IIS:\AppPools\CAS -Name managedRuntimeVersion -Value ''

# Verify - managedRuntimeVersion must be EMPTY (that is "No Managed Code")
Get-ChildItem IIS:\AppPools | Select-Object Name, managedRuntimeVersion, state

```

6.2 Application-pool advanced settings

Select the **CAS** pool, then **Advanced Settings...** in the Actions pane. The idle and recycle timers are deliberately **0** so the worker is never killed mid-request or mid-SignalR-stream; the pool recycles only on deployment.

IIS Manager > Application Pools > CAS > Advanced Settings...

Advanced Settings	
(General)	
.NET CLR Version	No Managed Code
Managed Pipeline Mode	Integrated
Start Mode	AlwaysRunning
CPU	
Limit Action	NoAction
Process Model	
Identity	BANK\svc_cas
Idle Time-out (minutes)	0
Load User Profile	True
Maximum Worker Processes	1
Recycling	
Regular Time Interval (minutes)	0
Disable Overlapped Recycle	True

Figure 6.2 — Advanced Settings for the **CAS** pool. Highlighted rows are the ones that matter for this application.

Setting	Value	Why
.NET CLR Version	No Managed Code	Standard for ANCM-hosted applications.
Identity	BANK\svc_cas	Domain service account — needed for LDAP integrated auth, NAS access and certificate private keys.
Idle Time-out (minutes)	0	The worker is never killed for inactivity; that would drop every SignalR connection.
Regular Time Interval (minutes)	0	Recycle only on deployment — never on a clock, which would interrupt long report generation.
Start Mode	AlwaysRunning	Warms the worker so the first request after a recycle is fast and background jobs resume immediately.
Maximum Worker Processes	1	Must stay 1. A web garden would run duplicate background jobs and break in-memory state.
Disable Overlapped Recycle	True	Prevents two workers holding the same log files and job leases during a recycle.
Load User Profile	True	Required for certificate and temp-file access under the service account.

Run on: each app node POWERSHELL EQUIVALENT

```
Import-Module WebAdministration
$pool = 'IIS:\AppPools\CAS'

Set-ItemProperty $pool -Name processModel.identityType -Value SpecificUser
Set-ItemProperty $pool -Name processModel.userName -Value 'BANK\svc_cas'
Set-ItemProperty $pool -Name processModel.password -Value (Read-Host 'svc_cas password')
Set-ItemProperty $pool -Name processModel.idleTimeout -Value '00:00:00'
Set-ItemProperty $pool -Name processModel.loadUserProfile -Value $true
Set-ItemProperty $pool -Name recycling.periodicRestart.time -Value '00:00:00'
Set-ItemProperty $pool -Name startMode -Value AlwaysRunning
Set-ItemProperty $pool -Name processModel.maxProcesses -Value 1
Set-ItemProperty $pool -Name recycling.disallowOverlappingRotation -Value $true

# Verify
Get-ItemProperty $pool | Select-Object name, startMode,
    @{n='identity';e={$_.processModel.userName}},
    @{n='idleTimeout';e={$_.processModel.idleTimeout}},
    @{n='recycleTime';e={$_.recycling.periodicRestart.time}}
```

6.3 Import the OAuth2 certificates and grant private-key access

The application is its own OAuth2 / OpenID Connect server (OpenIddict) and requires **two** RSA-2048 certificates — a **signing** certificate and an **encryption** certificate — imported into `LocalMachine\My` on **every** node, with **identical thumbprints**. Generation and rotation are covered in [docs/deployment/multi-server-deployment.md](#) §2.

Everything in this section can be done in the UI — import, permissions and reading the thumbprint. PowerShell equivalents are given after each part for anyone who prefers them; neither path is required.

6.3.1 Import the two certificates (UI)

Run ▶ `certlm.msc` ▶ **Personal** ▶ **Certificates** ▶ right-click ▶ **All Tasks** ▶ **Import...**

1. Press **Win+R**, type `certlm.msc` and press Enter. This opens **Certificates — Local Computer**. (Do **not** use `certmgr.msc` — that is the *current user* store, and the app pool cannot read certificates from it.)
2. Expand **Personal**, right-click **Certificates**, then **All Tasks** ▶ **Import...**
3. In the wizard: **Store Location** is already **Local Machine**. Browse to `CAS-Signing.pfx` — change the file-type filter to *Personal Information Exchange (*.pfx;*.p12)* or the file will not appear.
4. Enter the PFX password. Leave **Mark this key as exportable** **unticked** unless the bank's policy requires it — the application only needs to read the key.
5. Choose **Place all certificates in the following store** → **Personal**. Finish.
6. Repeat for `CAS-Encryption.pfx`.

Run on: each app node POWERSHELL EQUIVALENT

```
$pw = Read-Host 'PFX password' -AsSecureString
Import-PfxCertificate -FilePath .\CAS-Signing.pfx -CertStoreLocation Cert:\LocalMachine\My -Password $pw
Import-PfxCertificate -FilePath .\CAS-Encryption.pfx -CertStoreLocation Cert:\LocalMachine\My -Password $pw
```

6.3.2 Read the thumbprints (UI)

Double-click a certificate → **Details** tab → scroll to **Thumbprint**. The value shown is what goes into `appsettings.Production.json`.

Run on: each app node POWERSHELL EQUIVALENT — CLEAN, PASTE-SAFE OUTPUT

```
Get-ChildItem Cert:\LocalMachine\My |
    Where-Object Subject -like '*CollateralAppraisal*' |
    Select-Object Subject, Thumbprint, NotAfter
```

6.3.3 Grant the app pool Read on the private keys (UI — no command-line equivalent needed)

This is the step operators most often forget.

certlm.msc › Personal › Certificates › right-click the certificate › All Tasks › Manage Private Keys...



Figure 6.3 — Grant the app-pool identity **Read** on the certificate's private key. Repeat for the encryption certificate. Without this the application fails at start-up with `CryptographicException: Keyset does not exist`.

Run on: BOTH app nodes CONFIRM THE THUMBPRINTS MATCH

```
# Run on node 1 and node 2 and compare the two outputs character by character.
Get-ChildItem Cert:\LocalMachine\My |
  Where-Object Subject -like '*CollateralAppraisal*' |
  Sort-Object Subject | Format-Table Subject, Thumbprint -AutoSize
```

6.4 Create the two websites

6.4.1 The API site

IIS Manager › Sites › Add Website...

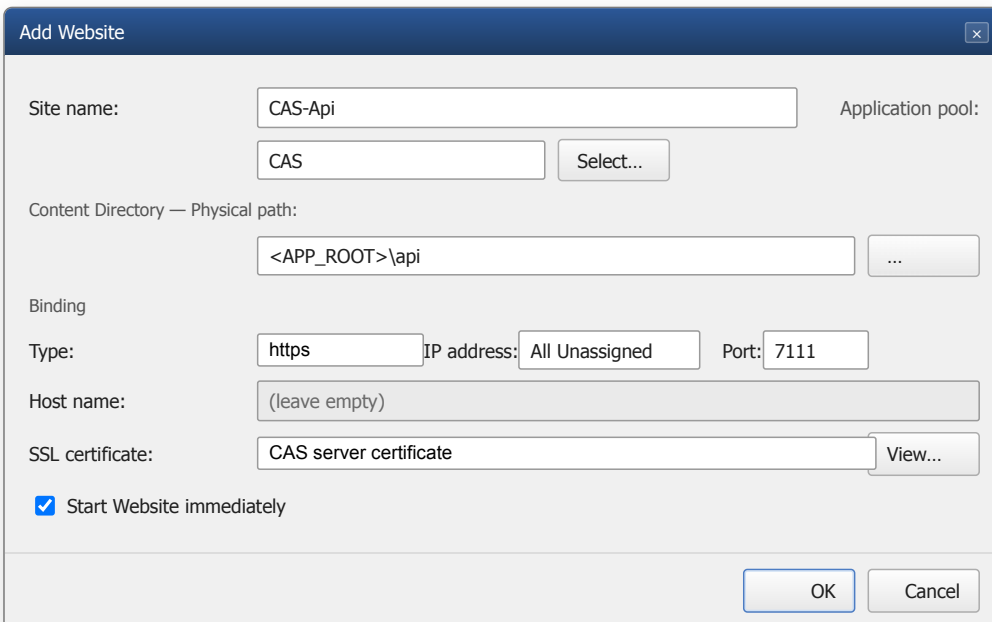


Figure 6.4.1 — The API site. Physical path `<APP_ROOT>\api`, application pool `CAS`, binding type `https` on port `7111`, with the server's TLS certificate selected. Host name is left empty — see the note below.

6.4.2 The SPA site

Add Website

Site name: Application pool:

Content Directory — Physical path:

Binding

Type: Port:

Host name:

SSL certificate:

Figure 6.4.2 — The SPA site. Physical path `<APP_ROOT>\web`, application pool `CAS`, binding type `https` on port `3000` — the same pool as the API site. Scheme and ports must match the URLs the SPA was built with and what the F5 publishes.

Run on: each app node **POWERSHELL EQUIVALENT**

```
New-Item -ItemType Directory -Path <APP_ROOT>\api, <APP_ROOT>\web, <APP_ROOT>\api\logs -Force

# The server TLS certificate must already be in LocalMachine\My. Grab its thumbprint:
$cert = Get-ChildItem Cert:\LocalMachine\My | Where-Object Subject -like '*cas*lhbank*'

New-Website -Name 'CAS-Api' -PhysicalPath '<APP_ROOT>\api' `
  -ApplicationPool 'CAS' -Port 7111 -Ssl
New-Website -Name 'CAS-Web' -PhysicalPath '<APP_ROOT>\web' `
  -ApplicationPool 'CAS' -Port 3000 -Ssl

# Bind the certificate to each port. 0.0.0.0 = all IPs, no host header.
New-Item -Path 'IIS:\SslBindings\0.0.0.0!7111' -Value $cert
New-Item -Path 'IIS:\SslBindings\0.0.0.0!3000' -Value $cert

# Verify
Get-Website | Select-Object Name, State, PhysicalPath, applicationPool,
  @{'n='bindings';e={$_.bindings.Collection | ForEach-Object bindingInformation} -join ' ' }
```

6.5 Raise the upload size limit

The application accepts document uploads up to **50 MB** (`FileStorage:MaxFileSizeBytes` = 52,428,800). IIS request filtering, however, rejects anything over its default of **30,000,000 bytes (~28.6 MB)** before the request ever reaches the application — the user sees a bare **HTTP 404.13** with no message from the app. Raise the limit on the **API** site.

IIS Manager > Sites > CAS-Api > Request Filtering > Edit Feature Settings...

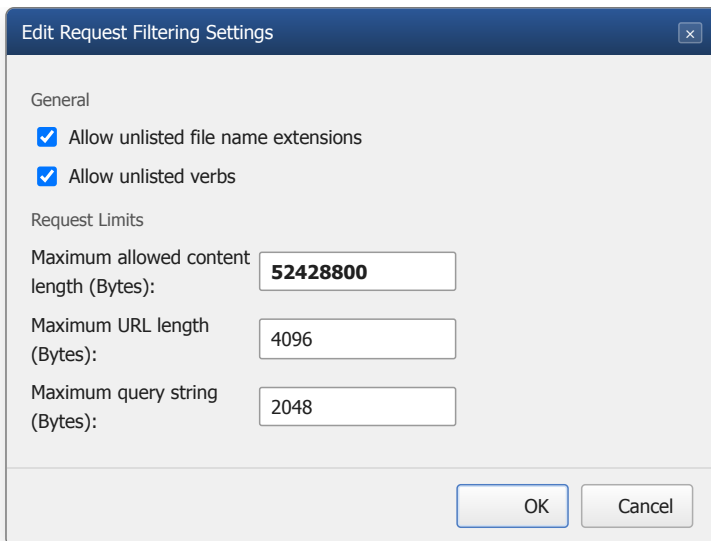


Figure 6.5 — Request Filtering. Set Maximum allowed content length to 52428800 (50 MB) on the API site.

Run on: each app node POWERSHELL EQUIVALENT

```
Set-WebConfigurationProperty -PSPath 'IIS:\Sites\CAS-API' `
  -Filter 'system.webServer/security/requestFiltering/requestLimits' `
  -Name maxAllowedContentLength -Value 52428800

# Verify
Get-WebConfigurationProperty -PSPath 'IIS:\Sites\CAS-API' `
  -Filter 'system.webServer/security/requestFiltering/requestLimits' `
  -Name maxAllowedContentLength | Select-Object Value
```

This setting lives in `web.config`, which the deploy script preserves. Set it once; it survives releases. If you ever recreate `web.config` from a fresh publish, re-apply it.

6.6 Confirm the API `web.config`

The publish output ships a `web.config` with `hostingModel="inprocess"`, the correct choice here — in-process passes the WebSocket upgrade straight to Kestrel. The server's own copy is preserved by the deploy script and never overwritten.

Run on: each app node INSPECT

```
Get-Content <APP_ROOT>\api\web.config

# Expect, inside <system.webServer>:
# <aspNetCore processPath="dotnet" arguments=".\\Api.dll"
#           stdoutLogEnabled="false" hostingModel="inprocess" />
# <requestLimits maxAllowedContentLength="52428800" /> (from §6.5)
```

Enabling the stdout log for troubleshooting. If the application will not start and the Event Log is not specific enough, temporarily set `stdoutLogEnabled="true"` and `stdoutLogFile=".\\logs\\stdout"`, reproduce the failure, then read the file and **set it back to false** — it grows without limit.

6.7 Verify the SPA site serves deep links

Run on: each app node POWERSHELL

```
# Root document
curl.exe -k -s -o NUL -w "%{http_code}\n" https://localhost:3000/ # 200

# A client-side route that is NOT a real file — proves the rewrite rule works
curl.exe -k -s -o NUL -w "%{http_code}\n" https://localhost:3000/appraisals # 200, NOT 404
```

A 404 on the second command means the URL Rewrite module is missing or `web.config` was not deployed into `<APP_ROOT>\web`.

7. Post-deployment verification

Health endpoints (anonymous)

Endpoint	Meaning
<code>/health/live</code>	The process is up. Nothing else is checked.
<code>/health/ready</code>	SQL + Redis + the message bus are ready — this is the F5 rotation gate . Must return 200 before a node takes traffic.
<code>/health/external</code>	LDAP / SMTP / SFTP status. Informational; a failure here does not pull the node out of rotation.
<code>/health</code>	Everything, with per-check detail. Use this when <code>/health/ready</code> is failing and you need to know which dependency.

Run on: each app node POWERSHELL

```
curl.exe -k -s -o NUL -w "%{http_code}\n" https://localhost:7111/health/ready
# Expect 200

# If it is not 200, get the detail:
curl.exe -k -s https://localhost:7111/health | ConvertFrom-Json |
    Select-Object -ExpandProperty checks |
    Format-Table name, status, exception
```

```
# Healthy output looks like:
name          status
----          -
sqlserver     Healthy
redis         Healthy
masstransit-bus Healthy
ldap          Healthy
smtp          Healthy
sftp          Healthy
```

8. Troubleshooting

8.1 Where to look first

Source	How to read it
Application log	<code>Get-Content <APP_ROOT>\api\logs\log-*.txt -Tail 100</code>
Windows Event Log (start-up failures)	<code>Get-EventLog -LogName Application -Source 'IIS AspNetCore Module V2' -Newest 20</code>
Health detail	<code>curl.exe -k https://localhost:7111/health</code>
IIS request log	<code>C:\inetpub\logs\LogFiles\W3SVC<id>\</code>
Database journal	<code>SELECT TOP 50 * FROM dbo.DatabaseMigrationHistory ORDER BY Applied DESC</code>

8.2 Symptom index

Symptom	Cause & fix
HTTP 500.19 or 500.31 from IIS	.NET 9 Hosting Bundle missing, or IIS was not reset after installing it. Reinstall the Hosting Bundle and run <code>iisreset</code> .
<code>CryptographicException: Keyset does not exist</code> at start-up	The app pool lacks Read on a certificate private key. Redo §6.3 for the certificate named in the stack trace.
<i>Production requires OAuth2:SigningCertificate...</i> and the app will not start	Working as intended. Fill both certificate thumbprints in <code>appsettings.Production.json</code> .
<code>IDX10503: Signature validation failed</code>	The nodes have different signing certificates. Re-import the signing PFX on the deviating node; confirm identical thumbprints (§6.3).
<code>IDX10609: Decryption failed</code>	The nodes have different encryption certificates. Same fix, encryption certificate.
<i>The antiforgery token could not be decrypted</i> after a deployment	Data Protection keyring problem: <code>auth.DataProtectionKeys</code> is empty (Step 2 not applied) or the nodes point at different databases. Apply the SQL scripts and restart both nodes.
Discovery document shows an internal hostname or <code>http://</code>	Forwarded headers not arriving. Ensure the F5 sends <code>X-Forwarded-Proto</code> and <code>X-Forwarded-For</code> (§6.4).
Endless redirect loop on sign-in	Same cause: HTTPS redirection sees <code>http</code> because <code>X-Forwarded-Proto</code> is missing.
SPA loads but every API call fails with a CORS error	<code>Cors:AllowedOrigins</code> does not exactly match the SPA origin, or the SPA was built with the wrong <code>VITE_API_URL</code> . Compare the browser console's origin with the config value (§Step 1).
SPA home page works, deep links return 404	URL Rewrite module not installed, or <code>web.config</code> absent from <code><APP_ROOT>\web</code> .
SignalR returns 200 then falls back to polling (no 101)	The Windows WebSocket Protocol feature is not enabled on that node. Realtime still works on the fallback transport; install the feature and recycle the pool to get the upgrade.
Realtime updates reach only some users	Redis backplane not shared: check <code>SignalR:UseRedisBackplane = true</code> and the same Redis endpoint and <code>ChannelPrefix</code> on both nodes.
Upload of a 40 MB file returns 404.13	IIS request limit still at the 30 MB default. Apply §6.5 on the API site.
Documents 404 when downloaded a second time	The nodes are not sharing storage: <code>FileStorage:Mode</code> must be <code>Nas</code> with the same <code>NasBasePath</code> on both.
Uploads fail with an access error	The app-pool service account lacks Modify on the NAS share. Test as that account, not as your admin account.
Report PDFs fail or hang	Chromium missing or <code>Reporting:ChromiumExecutablePath</code> wrong on that node. Install Edge/Chrome and correct the path.
Application starts, then fails with a pending-migration error	Step 2 was not applied for this release. Apply the SQL scripts, then restart the pool.
<code>03_OneTime_DataScripts.sql</code> fails partway	Read the log for the last <code>APPLY:</code> line — that is the failing script. Fix the cause, then re-run the whole file; everything already applied is skipped.
A view is missing after Step 2	Check <code>99_Verify.sql</code> section 4 for unresolved references, and search <code>02.log</code> for the view name.
Two Hangfire servers become four	Stale server registrations from a previous run; they expire automatically. If a node shows more than one, check Maximum Worker Processes = 1 (§6.2).
Files copied, but <code>/health/ready</code> never returns 200	The build deployed but the application is unhealthy. Open <code>https://localhost:7111/health</code> to see which dependency failed, and read the application log.

9. Appendix — ports

Ports

Port	Direction	Purpose
7111	F5 → node (inbound, HTTPS)	API site. Health gate <code>/health/ready</code> — the F5 monitor must use HTTPS.
3000	F5 → node (inbound, HTTPS)	SPA site (static files).
1433	node → SQL (outbound)	Application database, Hangfire store, Data Protection keys, OpenIddict store.
6379	node → Redis (outbound)	Distributed cache and SignalR backplane.
5672 / 15672	local to the node	RabbitMQ broker (co-located) and its management UI.
587	node → SMTP (outbound)	Outbound email (STARTTLS).
445	node → NAS (outbound)	SMB document and report storage.
22	node → SFTP (outbound)	AS400 reappraisal file exchange.
389 / 636	node → AD (outbound)	LDAP / LDAPS sign-in and directory lookups.

Collateral Appraisal System — Production Deployment Guide, version 2.0. Internal operations document. Verify every command, path, port and account name against the live release and the bank's own naming standards before executing in production.